



SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I Midhuna A (192521302) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A. Midhuna

Annexure A

SCENARIO-BASED TASK

QUESTIONS:

- 1.Scenario: Parsing Table Conflict in LL(1) While designing an LL(1) parser, a student constructs a parsing table and finds multiple entries in a single cell for a non-terminal and input symbol combination. This leads to confusion during parsing. Analyze how the syntax analyzer uses FIRST and FOLLOW sets in this process, identify the reason for the conflict, and explain how it affects deterministic parsing. Conclude by suggesting what decision should be taken to modify the grammar so that it becomes suitable for LL(1) parsing.
- 2.Scenario: Incorrect Parse Tree Generation A syntax analyzer generates a parse tree for the expression $id + id - id$, but the evaluation gives incorrect results due to improper grouping of operators. Analyze how the syntax analyzer constructs the parse tree, identify the issue related to associativity, and explain how grammar rules influence the structure of the tree. Finally, discuss what decision should be taken to enforce correct associativity and ensure accurate parsing.
- 3.Scenario: Invalid Expression Handling A syntax analyzer in a compiler receives the token sequence corresponding to the expression $a + * b$ from the lexical analyzer. Although all tokens are valid individually, the parser fails to generate a parse tree and reports an error. In this situation, analyze how the syntax analyzer identifies the error based on grammar rules, determine the key issue in the token arrangement, and explain how parsing techniques such as predictive or shift-reduce parsing handle such cases. Further, discuss what decision should be taken to improve error detection and recovery while maintaining clarity in parsing.
4. Scenario: Ambiguity in Conditional Statements A syntax analyzer is designed using the grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$. While constructing the parsing table, conflicts arise, and the parser cannot decide which production to apply for certain inputs. In this scenario, explain how the syntax analyzer encounters ambiguity, identify the key issue known as the dangling else problem, and discuss how it impacts parsing. Suggest an appropriate modification or parsing strategy to resolve the ambiguity and ensure correct association of else clauses.
- 5.Scenario: Operator Precedence Conflict A parser is designed for arithmetic expressions using the grammar $E \rightarrow E + E \mid E * E \mid id$. While parsing the input $id + id * id$, the syntax analyzer produces an incorrect parse tree due to ambiguity in operator precedence. Examine how the syntax analyzer processes this input, identify the issue caused by the grammar, and explain how precedence and associativity rules should be applied. Based on this, suggest what changes must be made to the grammar or parsing technique to ensure correct evaluation.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

CO2 Assessment -3
Scenario Based Task

name: A. Medhuna
Register no: 192521302
date: 7/8/26
course code: CSA1402
compiler design

1. Scenario: parsing Table conflict in LL(1)

In an LL(1) parser Syntax analyser construct parsing table using the FIRST and Follow sets of grammar. The FIRST set determines terminals that can appear at the beginning strings derived from a production, while Follow set contains terminals that can appear immediately after non-terminal during parsing table construction each production placed table placed sets.

First(α) symbols $\alpha \Rightarrow E$ then Follow(A)

$A \rightarrow \alpha | \beta$

$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) \neq \emptyset$

A conflict occurs when more than one production entered into same table cell nonterminal input symbol. This usually happens because grammar contains left recursion common prefix (left factoring problem) ambiguous cannot uniquely production should applied

Since LL(1) parsing deterministic every table entry must contain only one production multiple entries parser non determine incorrect parsing parser failure.

* multiple entries LL(1) parsing table all indicate the grammar not LL(1). The grammar should be modified using left factoring, left recursion elimination, etc. table cell one production.

2. Scenario: Incorrect parse tree Generation

The syntax analyzer constructs parse tree by applying grammar productions according to the input tokens. For the expression $id + id / id$, the parse tree determines order operations are evaluated.

If the grammar does not specify operator associativity, expression grouped incorrectly: $E \rightarrow E + E \mid E - E \mid id$

For input $id + id - id$, grammar generates two different interpretations: $(id + id) - id$

$id + (id - id)$ Grammar ambiguous, not specify + and - grouped

The operators + and - normally follow left to right (id + id) - id decision. The grammar should enforce left associativity for addition and subtraction: $E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow id$

This ensures expression evaluates left to right, producing correct parse tree and accurate result.

3. Scenario: Invalid Expression Handling

The lexical analyzer correctly identifies tokens in expression $a + * b$, identifies and operators. Syntax analyzer checks whether tokens follow grammar rules.

The error occurs because the grammar expects an operand after the + operator, but another operator appears *. Instead, the token sequence violates grammar, making

impossible to construct valid parse tree.

In predictive parsing (LL) the parser detects error when no valid production exists parsing table current input symbol.

In shift reduce parsing (LR) parser detects error when no valid shift or reduce action available.

without proper recovery parser stops after reporting error.

Decision: The parser should implement error detection recovery techniques panic mode recovery phrase level recovery or synchronization. Follow sets. The methods allow parser continue analyzing remaining input providing meaningful error messages.

4. Scenario: Ambiguity in conditional statements.

The grammar $S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow a$ & ambiguous because parser cannot determine if statement else belongs. This known dangling else problem. when constructing parsing table conflicts arise because both production begin same prefix:

If $E \text{ then } S$: The parser cannot decide whether reduce using first production or continue parsing the second production containing else.

This ambiguity affects deterministic parsing by introducing parsing conflicts producing multiple possible parse trees.

Decision: The grammar should be rewritten distinguish matched and unmatched statements for example:

$S \rightarrow M \mid U$

$M \rightarrow \text{if } E \text{ then } M \text{ else } M \mid a$

$U \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } M \text{ else } U.$

Alternatively parser generators often resolve the ambiguity association else with nearest unmatched if which is the

Standard rule used in most programming language.

5. Scenario: operator precedence conflict

The grammar: $E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow id$ is ambiguous because does not define operator precedence or associativity.

For input $id + id * id$ two different parse trees are possible

$* (id + id) * id$

$* id + (id * id)$

The correct evaluation requires multiplication have higher precedence than addition. Since the grammar does not specify rule, parser cannot determine correct parse tree.

operator precedence associativity essential ensure expression evaluated correctly

Decision: The grammar should be rewritten enforce precedence for example: $E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

* multiplication has higher precedence than addition

* Addition & multiplication left associative

Alternatively parser generators use precedence associativity declaration resolve conflicts automatically.